A good alternative to rounding is instead to test the significance of the difference between two floats. To do this we first need a special number, which is the system *epsilon*. This is the smallest floating-point number that satisfies *x + 1.0 != 1.0*. In other words, the epsilon is the smallest float we can add to 1.0 so that it no longer equals 1.0. This number will of course vary between 32-bit and 64-bit systems and is accessible via the built-in constant `PHP_FLOAT_EPSILON`.

One way to compare floats for equivalence (not equality) is to take the absolute value of their difference and compare to epsilon:

```
$a = 0.8;
$b = 0.1 + 0.7;
$absoluteDiff = abs($a - $b);

if ($absoluteDiff < PHP_FLOAT_EPSILON) {
    print "Float values are equivalent..." . PHP_EOL;
    var_dump(PHP_FLOAT_EPSILON);
    var_dump($absoluteDiff);
}

Output:
Float values are equivalent...
float(2.220446049250313E-16)
float(1.1102230246251565E-16)
```

Which is fine for tiny numbers but does not work well for very large floats:

```
$a = PHP_FLOAT_MAX;
//   1.7976931348623157E+308 = PHP_FLOAT_MAX (64bit)
$b = 1.7976931348623156E+308;

var_dump(abs($b - $a)); // absolute

Output:
float(1.99584030953472E+292)
```

Even though our two numbers are equivalent to the 15th significant digit, the absolute difference is in the order of 10^{292}! What we really need to do is compare the *relative difference* to epsilon, which factors in the magnitude of the comparison: